

Basic Deployment Pipeline



Outcome

- The ultimate purpose of all this is to get feedback as fast as possible
- To make the feedback cycle fast
- To be able to see which build is deployed into which environment and
- Which stages in your pipeline each build has passed

Note: It means that if you see a problem in the acceptance tests (for example), you can immediately find out which changes were checked into version control that resulted in the acceptance tests failing

BITS Pilani, Pilani Campus

Antipatterns



Antipatterns of dealing with Binaries

- The source code will be compiled repeatedly in different contexts: during the commit process, again at acceptance test time, again for capacity testing, and often once for each separate deployment target



BITS Pilani, Pilani Campus

Antipatterns of dealing with Binaries



Violates two Principles

Efficiency of Deployment pipeline

- Recompiling violates this principle because it takes time, especially in large systems
- Delayed feedback

Always build upon foundations

- The binaries that get deployed into production should be exactly the same as those that went through the acceptance test process
- Recreation of binaries violates this principle

BITS Pilani, Pilani Campus

Deployment Pattern Practices



1. Only Build Your Binaries Once
2. Deploy the Same Way to Every Environment
Note: Using the same script to deploy to production that you use to deploy to development environments is a fantastic way to prevent the “it works on my machine” syndrome
3. Smoke-Test Your Deployments
4. Deploy into a Copy of Production
5. Each Change Should Propagate through the Pipeline Instantly
6. Intelligent pipeline scheduling
7. If Any Part of the Pipeline Fails, Stop the Line

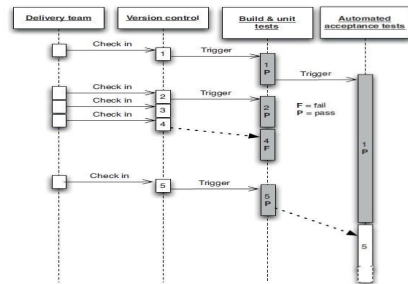
BITS Pilani, Pilani Campus



Deployment Pattern Practices

Intelligent scheduling

- Intelligent scheduling is crucial to implementing a deployment pipeline



Stages in Deployment Pipeline

- The Commit Stage
- The Automated Acceptance Gate
- Subsequent Test Stages
- Preparing to Release

BITS Pilani, Pilani Campus

Rolling Back Deployments and Zero-Downtime Releases

- It is essential to be able to roll back a deployment in case it goes wrong.
- Debugging problems in a running production environment is almost certain to result in late nights, mistakes with unfortunate consequences, and angry users
- Need to have a way to restore service to your users when things go wrong, so you can debug the failure in the comfort of normal working hours.
- more advanced techniques—blue-green deployments and canary releasing—can also be used to perform zero-downtime releases and rollbacks
- If release process makes changes to your data, it can be hard to roll back.
- Another constraint is the other systems you integrate with. With releases involving more than one system (known as orchestrated releases), the rollback process becomes more complex too.
- Two general principles you should follow when creating a plan for rolling back a release.
- The first is to ensure that the state of your production system, including databases and state held on the filesystem, is backed up before doing a release.
- The second is to practice your rollback plan, including restoring from the backup or migrating the database back before every release to make sure it works.

BITS Pilani, Pilani Campus

Rolling Back Deployments and Zero-Downtime Releases

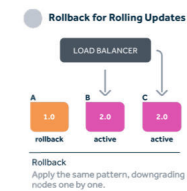
Patterns

- Rolling Back by Redeploying the Previous Good Version
- Rolling updates
- Zero-Downtime Releases
 - Blue-Green Deployments
 - Canary Releasing

BITS Pilani, Pilani Campus

Rolling Back by Redeploying the Previous Good Version

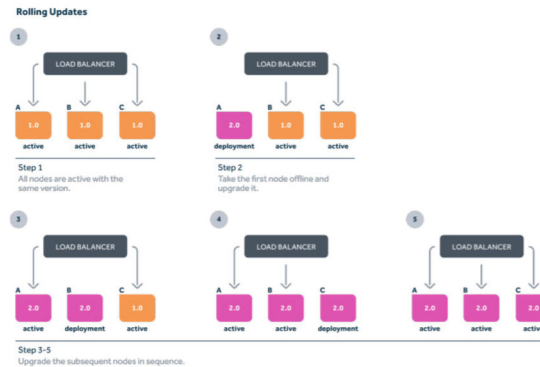
- This is often the simplest way to roll back. If you have an automated process for deploying your application, the simplest way to get back to a good state is to redeploy the previous good version from scratch.



BITS Pilani, Pilani Campus

Rolling Updates

- In a rolling update, each server node in the environment is taken offline and upgraded, one by one. After a node is upgraded, manual or automatic smoke testing determines whether the application is functioning as expected.



BITS Pilani, Pilani Campus

Zero-Downtime Releases

- A zero-downtime release, also known as hot deployment, is one in which the actual process of switching users from one release to another happens nearly instantaneously.
- Releasing software that guarantees that a service or application will not go offline at any point throughout the rollout process
- Strategies for Achieving Zero-Downtime Deployment
 - Blue-Green Deployments
 - Canary Releasing

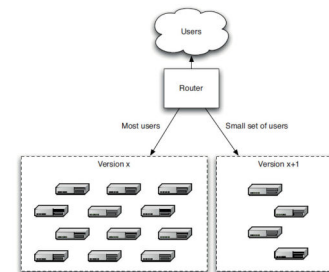
BITS Pilani, Pilani Campus

Blue-Green Deployments

- Blue/green deployments provide near zero-downtime release and rollback capabilities.
- The fundamental idea behind blue/green deployment is to shift traffic between two identical environments that are running different versions of your application. The blue environment represents the current application version serving production traffic.

BITS Pilani, Pilani Campus

Canary Releasing



- rolling out a new version of an application to a subset of the production servers to get fast feedback.
- Like a canary in a coal mine, this quickly uncovers any problems with the new version without impacting the majority of users

BITS Pilani, Pilani Campus

Deployment - Tips and Tricks

- The People Who Do the Deployment Should Be Involved in Creating the Deployment Process
- Things Go Better When Development and Operations Are Friends -> Go hand-in-hand
- Log Deployment Activities
- If your deployment process isn't completely automated, including the provisioning of environments, it is important to log all the files that your automated deployment process copies or creates
- Don't Delete the Old Files, Move Them
- Deployment Is the Whole Team's Responsibility
- A "build and deployment expert" is an antipattern - Every member of the team should know how to deploy, and every member of the team should know how to maintain the deployment scripts

Thank you

IMP Note to Self





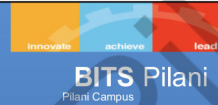
BITS Pilani
Pilani | Dubai | Goa | Hyderabad | Mumbai
**WORK INTEGRATED
LEARNING PROGRAMMES**



Introduction to DevOps

Slides edited /compiled by K V Vamsi Krishna
for DevOps faculty team.

IMP Note to Self



CSIW ZG515, Introduction to DevOps

CS 10 : DevOps – Continuous Deployment

IMP Note to Students

- ☐ It is important to know that just login to the session does not guarantee the attendance.
- ☐ Once you join the session, continue till the end to consider you as present in the class.
- ☐ IMPORTANTLY, you need to make the class more interactive by responding to Professors queries in the session.
- ☐ **Whenever Professor calls your number / name ,you need to respond, otherwise it will be considered as ABSENT**



Coverage

Continuous Deployment

Rolling Back Deployments and Zero-Downtime Releases
 Blue/Green Deployment
 Rolling Upgrade
 The Canary Release pattern

T1 - Chapter 7,
 R1 - Chapter 14

BITS Pilani, Pilani Campus

Deploying and Releasing Applications

- **Creating a Release Strategy**
- **The Release Plan**
- **The First Deployment**
- **Modeling Your Release Process and Promoting Builds**
- **Promoting Configuration**
- **Orchestration**
- **Deployments to Staging Environments**
- **Rolling Back by Redeploying the Previous Good Version**
- **Zero-Downtime Releases**
 - Blue-Green Deployments
 - Canary Releasing
- **Emergency Fixes**

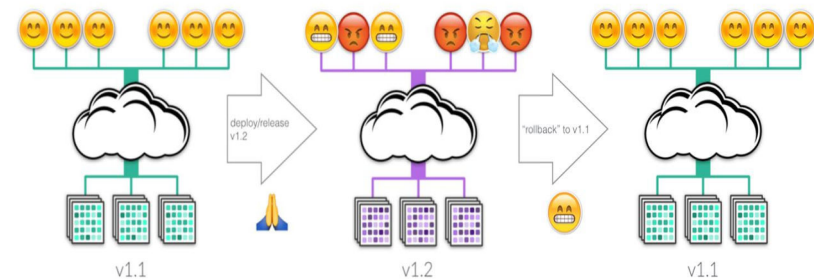
Rolling Back Deployments

Rollback Deployment

- In every deployment the consideration would be for some period the new version of service deployed will be in probation
- Even it has gone through testing, it is still not fully trusted
- Here comes the request for Rollback, which means reverting to the prior release
- It is even possible to roll forward which is just an instance of upgrading
- In Roll back it is highly recommended to have a human interference to decide "Whether the error is serious enough to justify the current deployment? Then "decide whether to roll back or roll forward"

BITS Pilani, Pilani Campus

Rollback



BITS Pilani, Pilani Campus

Zero Down Time Releases

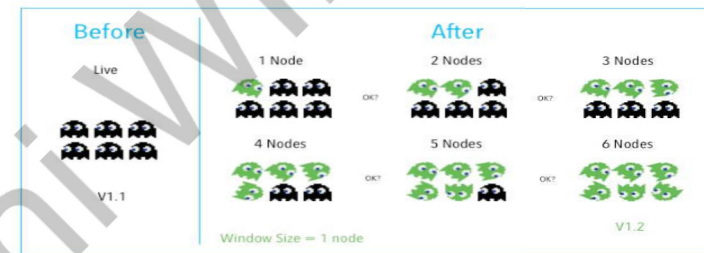
- Zero downtime releases empower your business to make changes while an application is running. Ideally, companies can use DevOps practices to take advantage of seamless deployment strategies that eliminate downtime for a live application and make sure the application is always up and running 24/7 even during the deployments.

BITS Pilani, Pilani Campus

Zero Down Time Deployment

Rolling Deployment

A rolling deployment is a deployment strategy that updates running instances of an application with the new release. All nodes in a target environment are incrementally updated with the service or artifact version in integer N batches.



BITS Pilani, Pilani Campus

Zero Down Time Deployment

Blue-Green Deployment

Blue-green deployment is a deployment strategy that utilizes two identical environments, a "blue" (aka staging) and a "green" (aka production) environment with different versions of an application or service. Quality assurance and user acceptance testing are typically done within the blue environment that hosts new versions or changes. User traffic is shifted from the green environment to the blue environment once new changes have been testing and accepted within the blue environment. You can then switch to the new environment once the deployment is successful.



BITS Pilani, Pilani Campus

Zero Down Time Deployment

Canary Deployment

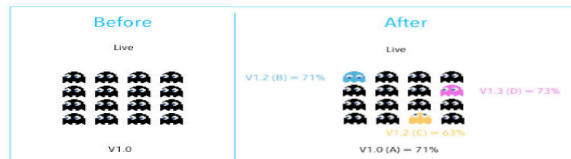
A canary deployment is a deployment strategy that releases an application or service incrementally to a subset of users. All infrastructure in a target environment is updated in small phases (e.g. 2%, 25%, 75%, 100%). A canary release is the lowest risk-prone, compared to all other deployment strategies, because of this control.



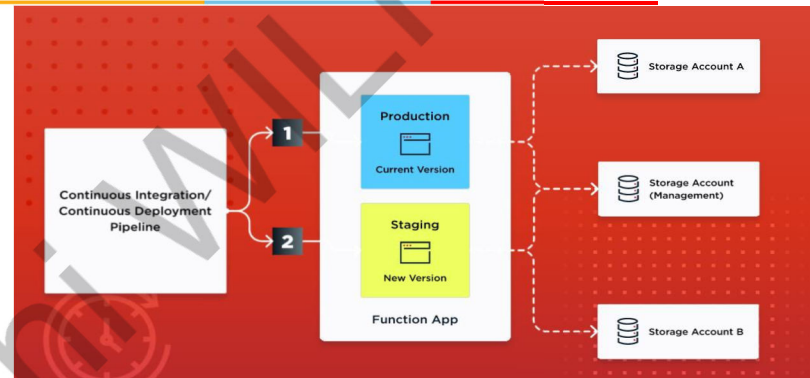


Zero Down Time Deployment

In A/B testing, different versions of the same service run simultaneously as "experiments" in the same environment for a period of time. Experiments are either controlled by feature flags toggling, A/B testing tools, or through distinct service deployments. It is the experiment owner's responsibility to define how user traffic is routed to each experiment and version of an application. Commonly, user traffic is routed based on specific rules or user demographics to perform measurements and comparisons between service versions. Target environments can then be updated with the optimal service version.



Blue Green Deployment



BITS Pilani, Pilani Campus

Blue Green Deployment

The Multi-Service Deployment

In a multi-service deployment, all nodes within a target environment are updated with multiple new services simultaneously. This strategy is used for application services that have service or version dependencies, or if you're deploying off-hours to resources that are not in use.



BITS Pilani, Pilani Campus

Blue Green Deployment

The Basic Deployment

In a basic deployment, all nodes within a target environment are updated at the same time with a new service or artifact version. Because of this, basic deployments are outage-proof and they slow down rollback processes or strategies. Of all the deployment strategies shared, it is the riskiest.



BITS Pilani, Pilani Campus



Blue Green Deployment

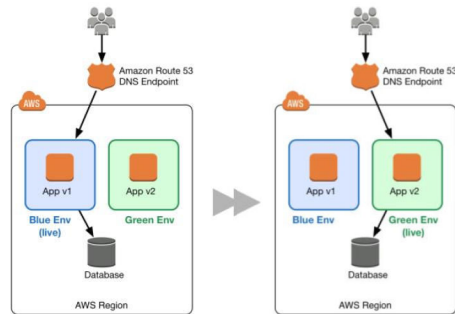


Figure 1: Basic blue/green example

BITS Pilani, Pilani Campus

Blue Green Deployment

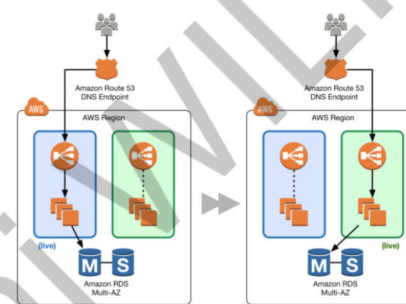


Figure 2: Classic DNS pattern

BITS Pilani, Pilani Campus

Blue Green Deployment

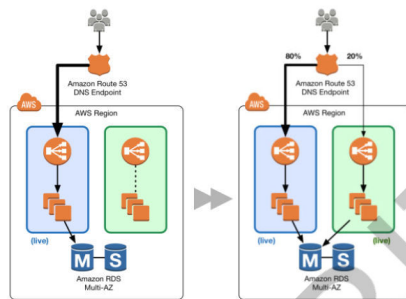


Figure 3: Classic DNS-weighted distribution

BITS Pilani, Pilani Campus

Blue Green Deployment

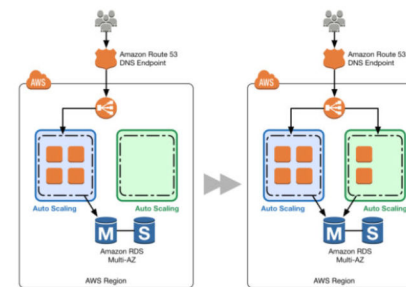


Figure 4: Swap Auto Scaling group pattern

BITS Pilani, Pilani Campus

Blue Green Deployment

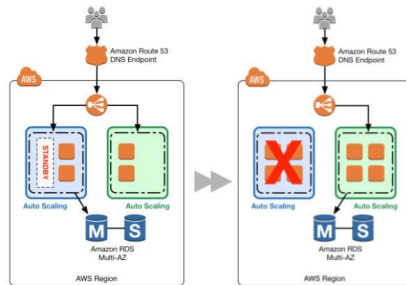


Figure 5: Blue Auto Scaling group nodes in standby and decommission

BITS Pilani, Pilani Campus

Blue Green Deployments - Challenges

1. **Infrastructure Cost:** Need to keep 2 separate instances of your infrastructure up at the same time, at least for the duration of the deployment but possibly for more time to allow for old sessions to drain or a quick rollback to the previous version.
2. **Stateful Applications:** If Blue-Green is used to deploy stateful applications, transient information that is stored in the instance (like user sessions, cached files, etc.) might be lost when traffic switches to the Green environment.
3. **Database changes:** Blue-Green deployments to database changes, this is usually not implemented due to the complexity of managing data migrations and the amount of data that may need to be transferred from one environment to another.
4. **Forward compatibility:** the old version of the application must be able to work with data created by the new version. This usually means that the old version must be able to handle extra fields in a database table or event schema without crashing.

BITS Pilani, Pilani Campus

"Big Bang" Deployment

Characteristics of big bang deployment include:

- All major pieces packaged in one deployment;
- Largely or completely replacing an existing software version with a new one;
- Deployment usually resulting in long development and testing cycles;
- Assuming a minimal chance of failure as rollbacks may be impossible or impractical;
- Completion times are usually long and can take multiple teams' efforts;
- Requiring action from clients to update the client-side installation.

BITS Pilani, Pilani Campus

"Big Bang" Deployment

Big bang deployments aren't suitable for modern applications because the risks are unacceptable for public-facing or business-critical applications where outages mean huge financial loss. Rollbacks are often costly, time-consuming, or even impossible.

The big bang approach can be suitable for non-production systems (e.g., re-creating a development environment) or vendor-packaged solutions like desktop applications.

BITS Pilani, Pilani Campus

Rolling Deployment

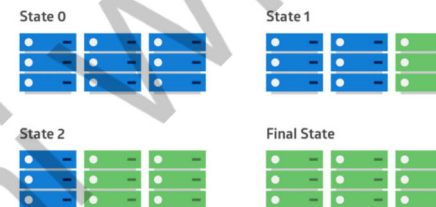
In a rolling deployment, an application's new version gradually replaces the old one. The actual deployment happens over a period of time. During that time, new and old versions will coexist without affecting functionality or user experience. This process makes it easier to roll back any new component incompatible with the old components.

An application suite upgrade is an example of a rolling deployment. If the original applications were deployed in containers, the upgrade can tackle one container at a time. Each container is modified to download the latest image from the app vendor's site. If there is a compatibility issue for one of the apps, the older image can recreate the container. In this case, the new and old versions of the suite's applications coexist until every app is upgraded.

BITS Pilani, Pilani Campus

Rolling Deployment

The following diagram shows the deployment pattern: the old version is shown in blue and the new version is shown in green across each server in the cluster.



BITS Pilani, Pilani Campus

Rolling Upgrade

- A Rolling Upgrade consists of deploying a small number of new version systems at a time directly to the production environment
- In this approach simultaneously you turn off the old version system
- Before you remove the original system make sure the new version is up and running
- Benefits
 - Cost Effective
 - Risk Effective

BITS Pilani, Pilani Campus

Rolling Upgrade

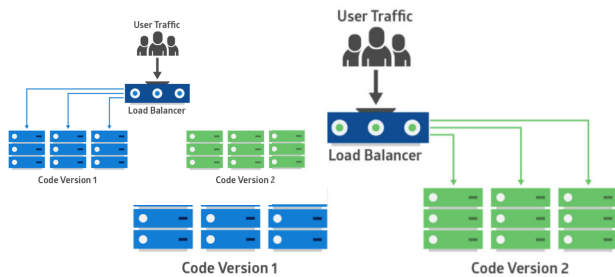
One is the currently-running production environment receiving all user traffic (depicted as Blue). The other is a clone of it, but idle (Green). Both use the same database back-end and app configuration:

The new version of the application is deployed in the green environment and tested for functionality and performance. Once the testing results are successful, application traffic is routed from blue to green. Green then becomes the new production.

In a blue-green deployment, both systems use the same persistence layer or database back end. It's essential to keep the application data in sync, but a mirrored database can help achieve that.



Rolling Upgrade



BITS Pilani, Pilani Campus

Rolling Upgrade

- Are your schema changes too complex to decouple from the code changes? Is sharing of data stores not feasible?
- Does your application need to be “deployment aware”?
- Does your commercial off-the-shelf (COTS) application come with a predefined update/upgrade process that isn't blue/green deployment friendly?

BITS Pilani, Pilani Campus

Canary Releasing

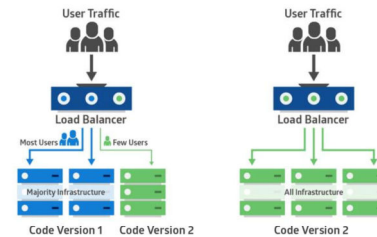
- of an application (Canary Deployment)
 - Subset of infrastructure as canaries
 - Incremental upgrade
- of a feature (Canary Release)
 - Subset of users as canaries
 - Incremental toggle
- Monitoring



BITS Pilani, Pilani Campus

Canary Deployment

With canary deployment, you deploy a new application code in a small part of the production infrastructure. Once the application is signed off for release, only a few users are routed to it. This minimizes any impact.



BITS Pilani, Pilani Campus

Canary Deployment

Consider a way to route new users by exploring several techniques:

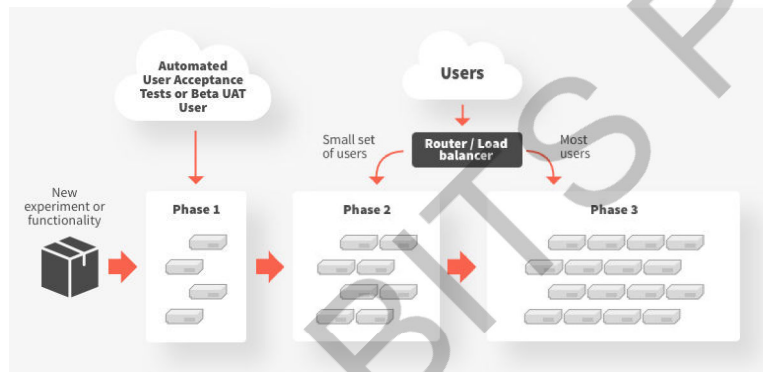
- Exposing internal users to the canary deployment before allowing external user access;
- Basing routing on the source IP range;
- Releasing the application in specific geographic regions;
- Using an application logic to unlock new features to specific users and groups. This logic is removed when the application goes live for the rest of the users.

BITS Pilani, Pilani Campus

Canary Releasing

- Safe assumption that to only have one version of a software in production at a time, this makes it much easier to manage bug fixes and indeed infrastructure in general
- It also presents a weakness to testing a software defects pop up in production
- In an extremely large production environment, its impossible to create a meaningful capacity testing environment
- How to ensure a new version of an application won't perform poorly
- Canary Releasing aims to address these challenges
- It involves rolling out a new version of an application to a subset of the production servers to get fast feedback
- Like a canary in a coal mine this quickly uncovers any problems with the new version without impacting the majority of users

Canary Releasing



Benefits

- It makes rolling back easy: Just stop routing users to the bad version and you can investigate the logs at your freedom
- One can use it for A/B testing by routing some users to the new version and some to the old version
- We can check the non functional requirements were met by gradually ramping up the load, slowly routing more users and measuring the non functional requirements and metrics like CPU usage, I/O and memory usage and watching for exceptions in logs



IMP Note to Self



Thank you



BITS Pilani
Pilani | Dubai | Goa | Hyderabad | Mumbai
**WORK INTEGRATED
LEARNING PROGRAMMES**



Introduction to DevOps

Slides edited /compiled by Dr. Suhaas KP
for DevOps faculty team.

IMP Note to Self



CSIW ZG515, Introduction to DevOps CS 11 : Continuous Monitoring

IMP Note to Students

- It is important to know that just login to the session does not guarantee the attendance.
- Once you join the session, continue till the end to consider you as present in the class.
- IMPORTANTLY, you need to make the class more interactive by responding to Professors queries in the session.
- **Whenever Professor calls your number / name ,you need to respond, otherwise it will be considered as ABSENT.**

BITS Pilani, Pilani Campus



Coverage

- Need for continuous monitoring
- Goals of monitoring
- Challenges of monitoring under continuous change
- Alert management
- Analytics
- Continuous customer feedback and optimization
- Overview of ELK (Elasticsearch, Logstash, and Kibana) Stack

BITS Pilani, Pilani Campus

DevOps Life Cycle Process

(Continuous Integration, Continuous Inspection, Continuous Deployment, Continuous Delivery, **Continuous Monitoring**)



BITS Pilani, Deemed to be University under Section 3 of UGC Act, 1956

What is Continuous Monitoring?

- Automated process by which **DevOps** personnel can observe and detect compliance issues and security threats during each phase of the DevOps pipeline.
- Outside DevOps, the process may be expanded to do the same for any segment of the IT infrastructure in question.
- It helps teams or organizations monitor, detect, study key relevant metrics, and find ways to resolve said issues in real time.
- Continuous Monitoring comes in at the **end of the DevOps pipeline**.

BITS Pilani, Deemed to be University under Section 3 of UGC Act, 1956

What is Continuous Monitoring?

- Once the software is released into production, Continuous Monitoring will **notify dev and QA teams** in the event of specific issues arising in the *prod* environment.
- It provides **feedback** on what is going wrong, which allows the relevant people to work on necessary fixes as soon as possible.
- This is especially helpful with implementing and fortifying various **security measures** – **incident response, threat assessment, computers, and database forensics, and root cause analysis**

BITS Pilani, Deemed to be University under Section 3 of UGC Act, 1956

Goals of Continuous Monitoring in DevOps

- **Enhance transparency** and visibility of IT and network operations, especially those that can trigger a security breach, and resolve it with a well-timed alert system.
- **Help monitor software operation**, especially performance issues, identify the cause of the error and apply appropriate solutions before significant damage to uptime and revenue.
- **Help track user behavior**, especially right after an update to a particular site or app has been pushed to prod. This monitors if the update has a positive, negative, or neutral effect on user experience.

BITS Pilani, Deemed to be University under Section 3 of UGC Act, 1956

What to Monitor?

- The data to be monitored for the most part comes from the various levels of the stack.
- The whole stack is involved in most of the purposes for which monitoring is required.

Goal of Monitoring	Source of Data
Failure detection	Application and infrastructure
Performance degradation detection	Application and infrastructure
Capacity planning	Application and infrastructure
User reaction to business offerings	Application
Intruder detection	Application and infrastructure

BITS Pilani, Deemed to be University under Section 3 of UGC Act, 1956

Failure Detection

- Detecting software failures can be done in one of three fashions.
 1. The monitoring software performs health checks on the system from an external point.
 2. A special agent inside the system performs the monitoring.
 3. The system itself detects problems and reports them

BITS Pilani, Deemed to be University under Section 3 of UGC Act, 1956

Performance Degradation Detection

- Degraded performance can be observed by comparing current performance to historical data—or by complaints from clients or end users.
- Ideally the monitoring system catches performance degradation before users are impacted at a notable strength.
- Latency
- Throughput
- Utilization

BITS Pilani, Deemed to be University under Section 3 of UGC Act, 1956

Capacity Planning

Long-Term Capacity Planning

- Long-term capacity planning is intended to match hardware needs, whether real or virtualized, with workload requirements.
- In a physical datacenter, it involves ordering hardware. In a virtualized public datacenter, it involves deciding on the number and characteristics of the virtual resources that are to be allocated

Short-Term Capacity Planning

- In the context of a virtualized environment such as the cloud, short-term capacity planning means creating a new virtual machine (VM) for an application or deleting an existing VM.
- A common method of making and executing these decisions is based on monitoring information collected by the infrastructure

BITS Pilani, Deemed to be University under Section 3 of UGC Act, 1956

User Interaction

User satisfaction depends on four elements that can be monitored.

1. The latency of a user request
2. The reliability of the system with which the user is interacting
3. The effect of a particular business offering or user interface modification
4. The organization's particular set of metrics.

Two types of user interaction monitoring-

1. Real user monitoring (RUM)
2. Synthetic monitoring

BITS Pilani, Deemed to be University under Section 3 of UGC Act, 1956

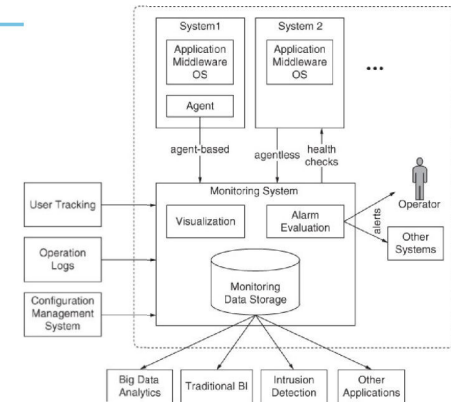
Intrusion Detection

- Intruders can break into a system by subverting an application, for example, through incorrect authorization or a man-in-the-middle attack.
- Applications can monitor users and their activities to determine whether the activities are consistent with the users' role in the organization or their past behavior.
- An intrusion detector is a software application that monitors network traffic by looking for abnormalities.
- These abnormalities can be caused either by attempts to compromise a system by unauthorized users or by violations of an organization's security policies.

BITS Pilani, Deemed to be University under Section 3 of UGC Act, 1956

How to Monitor?

- If the system is actively contributing to the data being monitored (the arrow labeled "agentless") then the monitoring is intrusive and affects the system design.
- If the system is not actively contributing to the data being monitored (the arrow labeled "agent-based") then the monitoring is nonintrusive and does not affect the system design.
- A third source of data is indicated by the arrow labeled "health checks."



BITS Pilani, Deemed to be University under Section 3 of UGC Act, 1956

When to Change the Monitoring Configuration?

Some examples of events that could change the monitoring configuration are-

- An alert
- Deployment
 - ✓ Canary deployment
 - ✓ Rolling upgrade
 - ✓ Feature activation or deactivation
- Changes to any infrastructure software including DevOps tools
- Changes to any configuration parameters

BITS Pilani, Deemed to be University under Section 3 of UGC Act, 1956

Interpreting Monitoring Data

Logs

- A log is a time series of events, since it is a sequence of records ordered by time.
- Logs usually record the actions performed that may result in a state change of the system.
- Logs should have a consistent format, should include an explanation for why this particular log message was produced and should include context information.

Graphing and Display

- It is useful to visualize it in various ways. Most monitoring data is time series data, which is amenable to plotting.

BITS Pilani, Deemed to be University under Section 3 of UGC Act, 1956

Interpreting Monitoring Data

Alarms and Alerts

- Monitoring systems inform the operator of significant events. This information can be in the form of either an alarm or an alert.
- Alarms and alerts are generated based on configurations set by the operators. Alarms and alerts can be triggered by events, by values crossing a threshold, or by sophisticated combinations of values and trends.
- Alarms can not only go off if something happens, they can also be set to go off if an expected event did not happen.

Diagnosis and Reaction

- Operators often use monitoring systems to diagnose the causes and observe the progress of mitigation and recovery

BITS Pilani, Deemed to be University under Section 3 of UGC Act, 1956

Challenges

Challenge 1: Monitoring Under Continuous Changes

- In a large-scale complex environment, *changes are the norm*.
- The new challenges come from both cloud elasticity, making infrastructure resources more volatile, and the automated DevOps operations, which trigger various sporadic operations (such as upgrade, reconfiguration, or backups).
- Sporadic operations and continuous deployment and deployment practices make software changes more frequent.
- Another challenge related to monitoring under continuous changes is the specification of monitoring parameters.
- Continuous changes in the system infrastructure and the system itself complicate the setting of monitoring parameters.

BITS Pilani, Deemed to be University under Section 3 of UGC Act, 1956

Challenges

Challenge 2: Bottom-Up vs. Top-Down and Monitoring in the Cloud

- Ideally, errors in lower layers and in individual components can be detected early, before they propagate and affect upper-layer application servers or applications themselves in terms of aggregated values. There are two challenges here-
 - First, there is usually a lot more to be monitored at the individual component level and other low levels.
 - A second challenge is related to continuous change caused by cloud elasticity and automation.
 - In the cloud, lower-layer infrastructure and servers come and go for both legitimate reasons, (e.g., scaling out/in, and rolling upgrades) as well as illegitimate reasons (e.g., instance failures or resource sharing uncertainty)

BITS Pilani, Deemed to be University under Section 3 of UGC Act, 1956

Challenges

Challenge 3: Monitoring a Microservice Architecture

- If any of the services is slow to respond, the overall response time will suffer.
- In a large-scale system, one part or another may experience some slowdown at any given time, which may lead to a negative impact on an unacceptable portion of the overall requests.
- Micropartitions and selective replication enable easier migration, which can be used to move services away from problematic parts of the network.
- Another challenge that microservice architectures raise is how to identify and fix “slow” nodes.

BITS Pilani, Deemed to be University under Section 3 of UGC Act, 1956

Challenges

Challenge 4: Dealing with Large Volumes of Distributed (Log) Data

- A large-scale system can easily generate millions of events, metric measurements, and log lines per minute. Some considerations about this volume of data are-
 - The performance overhead of collecting metrics at a small time interval might be significant.
 - Should use a modern distributed logging or messaging system for data collection, rather than building one yourself.
 - A distributed logging system such as **Logstash** can collect all kinds of logs and conduct a lot of local processing before shipping the data off.
 - With the emergence of big data analytics, researchers are starting to use advanced machine learning algorithms to deal with noisy, inconsistent, and voluminous data.

BITS Pilani, Deemed to be University under Section 3 of UGC Act, 1956

Types of Continuous Monitoring in DevOps

Infrastructure Monitoring:

- Monitors and manages the IT infrastructure required to deliver products and services.
- This includes data centers, networks, hardware, software, servers, storage, and the like.
- Infrastructure Monitoring collates and examines data from the IT ecosystem to improve product performance as far as possible.
- **Tools- Nagios, Prometheus**

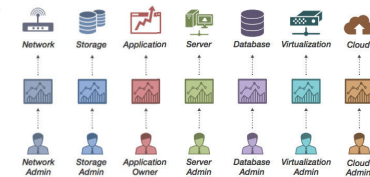


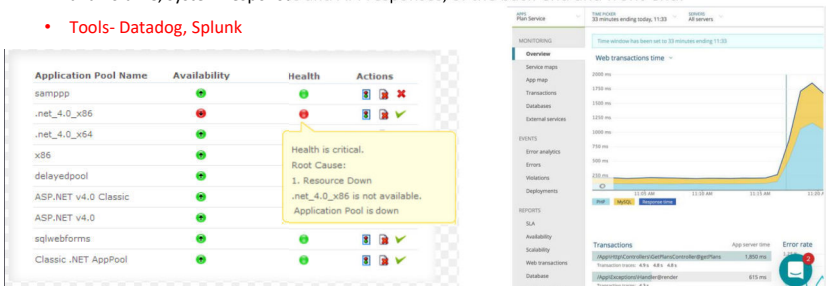
Figure 1: IT monitoring tool sprawl in organizations

BITS Pilani, Deemed to be University under Section 3 of UGC Act, 1956

Types of Continuous Monitoring in DevOps

Application Monitoring:

- Monitors the performance of released software based on metrics like uptime, transaction time and volume, system responses and API responses, of the back-end and front-end.
- Tools- Datadog, Splunk**



BITS Pilani, Deemed to be University under Section 3 of UGC Act, 1956

Types of Continuous Monitoring in DevOps

Network Monitoring:

- Monitors and tracks network activity, including the status and functioning of firewalls, routers, switches, servers, Virtual Machines, etc.
- Network Monitoring detects possible and present issues and alerts the relevant personnel.
- Its primary goal is to prevent network downtime and crashes
- Tools- Wireshark, NMap**

BITS Pilani, Deemed to be University under Section 3 of UGC Act, 1956

Benefits of Continuous Monitoring

Better Network Visibility and Transparency:

- CM offers DevOps teams clarity on the state of the IT infrastructure by automatically collecting and analyzing data to reflect possible outages and important trends.

Facilitates Rapid Responses:

- A primary aspect of CM is implementing an alert system that immediately notifies the right people the minute an IT incident emerges. This enables timely response to security threats or functional stop-gaps, minimizing damage and allowing faster restoration of the system to optimal operational levels.

Minimizes System Downtime:

- Consistent system monitoring and quick, necessary alerts help maintain system uptime by raising the alarm when there is a service outage or any application performance issues.

BITS Pilani, Deemed to be University under Section 3 of UGC Act, 1956

Benefits of Continuous Monitoring

Assists with Healthy Business Performance:

- Reduction in system downtime also minimizes negative impact on customer experience, thus safeguarding the organization against losses in revenue or credibility.
- As mentioned before, Continuous Monitoring tools can also be used to track user reactions to software updates, which is useful for several teams – development, QA, sales, marketing, customer service, etc.

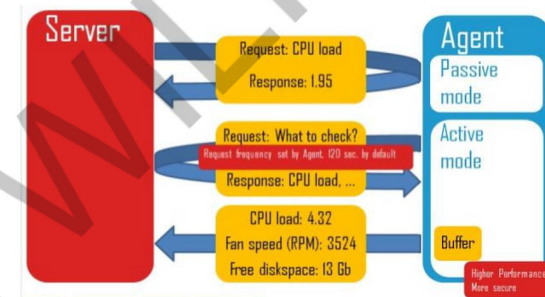
BITS Pilani, Deemed to be University under Section 3 of UGC Act, 1956

Zabbix: Monitoring Agent

- Zabbix is an open-source software tool to monitor IT infrastructure such as networks, servers, virtual machines, and cloud services.
- Zabbix collects and displays basic metrics.
- Active vs. Passive mode-



Zabbix: Monitoring Agent in Passive/Active Mode



Monitoring Coverage on System Stack

Resources	Experience	Web Analytics	Split Testing
	Software	Application	Availability
	Middleware	Registered users	Requests
	OS	Cache size	Process
Hardware	Process	Swapping	free space
	CPU	I/O	Memory
	Packet loss	Latency	Bandwidth use
	Network		

Figure 2-1. Monitoring coverage of system stack

Nagios: Monitoring Tool Study

- An open-source system and network application.
- Used to-
 - Monitor entire IT infrastructure
 - Spot problems before they occur
 - Know immediately when problems arise
 - Detect security breaches
 - Plan and budget for IT upgrades

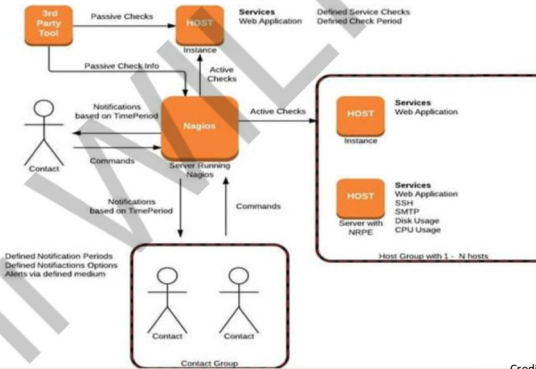


Nagios: Features

- Monitoring of network services
 - SMTP
 - POP3
 - HTTP
 - PING and more
- Monitoring of host resources
 - Processor load
 - Disk usage and more
- Simple plugin design that allows users to easily develop their own service checks
- Parallelized service checks

Credits: Nagios, Getting started by Hitesh Bhatia

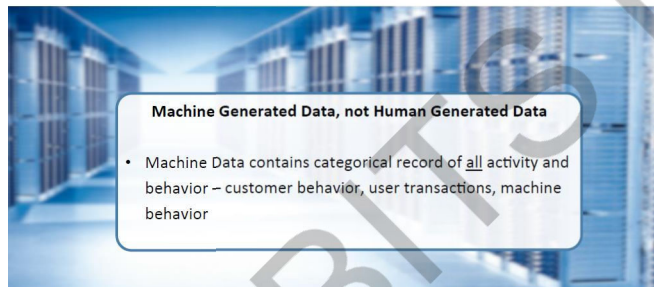
Nagios: The server and Services



Credits: Nagios, Getting started by Hitesh Bhatia

Splunk: Monitoring Tool Study

What does Splunk collect?



Splunk: Monitoring Tool Study

What does Splunk do?



What does Splunk Provide?



Splunk: Features

Data Ingestion

- Splunk can ingest a variety of data formats like JSON, XML and unstructured machine data like web and application logs.

Data Indexing

- The ingested data is indexed by Splunk for faster searching and querying on different conditions.

Data Searching

- Searching in Splunk involves using the indexed data for the purpose of creating metrics, predicting future trends and identifying patterns in the data.

Using Alerts

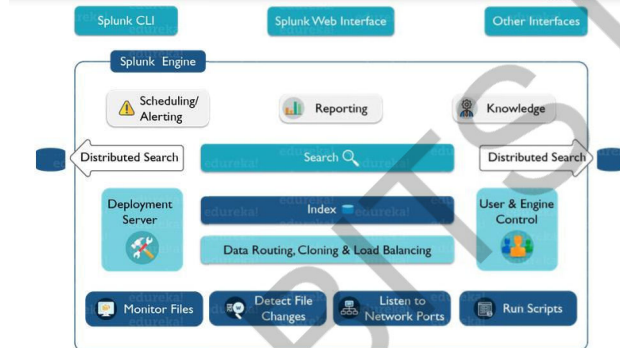
- Splunk alerts can be used to trigger emails or RSS feeds when some specific criteria are found in the data being analyzed. Splunk Dashboards can show the search results in the form of charts etc.

Splunk: Overview

Index all data without parsers or connectors



Splunk: Overview

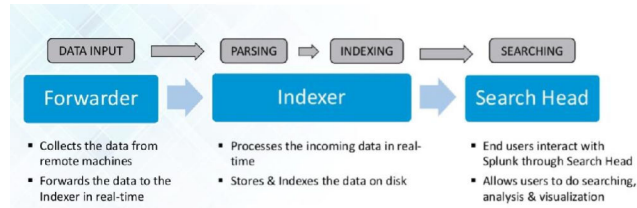


Splunk: Use case

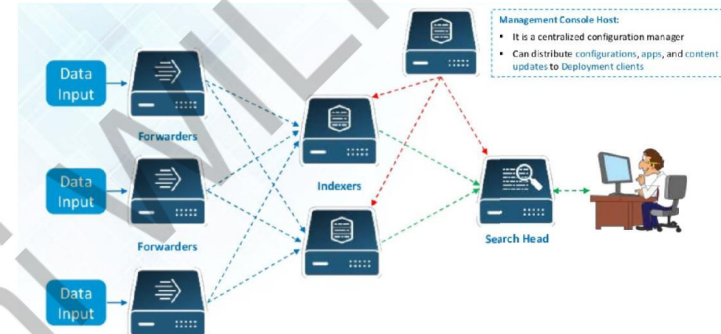




Splunk: Components

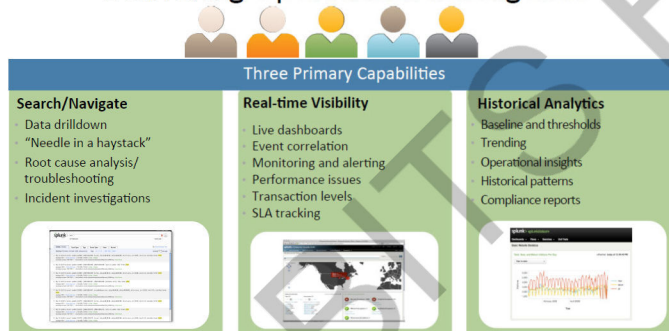


Splunk: Architecture



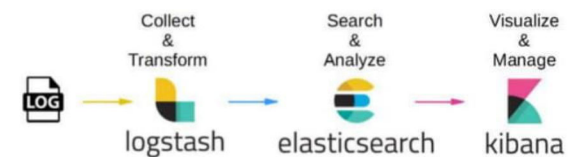
Splunk: Capabilities

Delivering Operational Intelligence

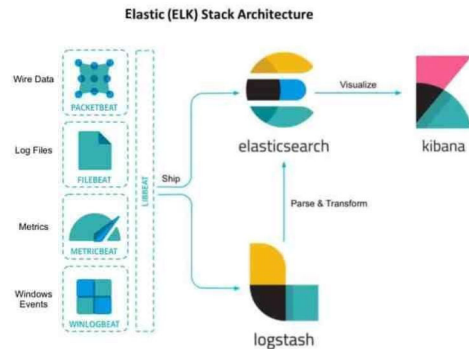


Elastic Stash (ELK)

- Elastic Stash is a group of open source products from Elastic designed to help users take data from any type of source and in any format and search, visualize that data in real time.
- It uses- **Logstash** for log aggregation, **Elasticsearch** for searching, **Kibana** for visualizing and analyzing data.



ELK Architecture



Elasticsearch

Server environment for storing large scale structured index entries and query them

- Written in Java
- Based on Apache Lucene
- Uses Lucene for index creation and management
- Document-oriented (structured) index entries which can (but must not) be associated with a schema
- Combines “full text”-oriented search options for text fields with more precise search options for other types of fields, like date + time fields, geolocation fields, etc.
- Near real-time search and analysis capabilities

Provides Restful API as JSON over HTTP

Elasticsearch- Github Case study

Challenge : How do you satisfy the search needs of GitHub's **4 million users** while simultaneously providing tactical operational insights that help you iteratively improve customer service?

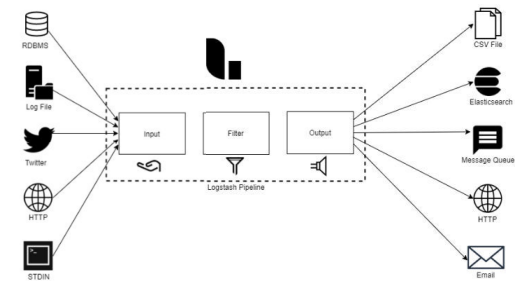
"Search is at the core of GitHub"

Solution: GitHub uses Elasticsearch to continually index the data from an ever-growing store of over **8 million code repositories**, comprising over **2 billion documents**. GitHub uses Elasticsearch to index new code as soon as users push it to a repository on GitHub.

Other customers includes Facebook, Netflix, ebay, Wikimedia, etc.
 ebay : Searching across 800 million listings in subseconds

Logstash

- Logstash is typically used as the “processing” engine for any log management solution (or systems that deal with changing data streams).
- These applications collect logs from different sources (software, hardware, electronic devices, API calls, etc.), process the collected data, and forwards it to a different application for further processing or storing.



Logstash: Plugins

Logstash has a rich collections of input, filter and output plugins. You can now create your own Logstash plugin and add it into community plugins.

Input Plugins

- Beats
- Elasticsearch
- File
- Graphite
- Heartbeat
- Ttp
- Jdbc
- Kafka
- Log4j
- Redis
- Stdin
- TCP
- Twitter

Filter Plugins

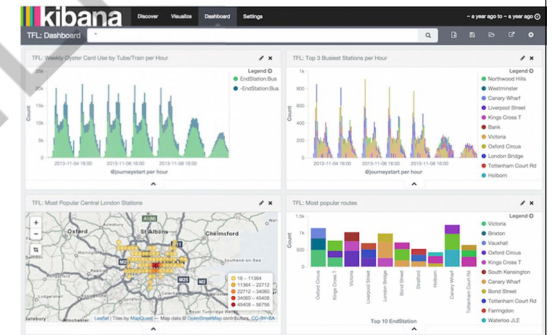
- Aggregate
- csv
- Date
- geoip
- Grok
- Json
- sleep
- urlencode
- UUID
- xml

Output Plugins

- CSV
- Elasticsearch
- Email
- File
- Graphite
- Http
- Jira
- Kafka
- Nagios
- Redis
- Stdout
- S3
- Tcp

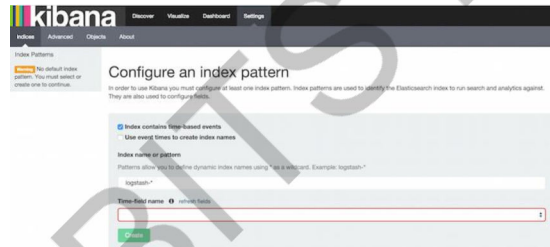
Kibana

- Web-based application for exploring and visualizing data
- Modern Browser-based interface (HTML5 + JavaScript)
- Ships with its own web server for easy setup
- Seamless integration with Elasticsearch



Kibana: Configure

- After installation first configure Kibana to access Elasticsearch server(s)
- Should be done by editing the Kibana **config file**
- Then use web UI to configure indexes to use



Kibana: Create visualization



Kibana: Visualization different types

Discover Visualize Dashboard Settings

Create a new visualization Step 1

	Area chart Great for stacked timelines in which the total of all series is more important than comparing any two or more series. Less useful for assessing the relative change of unrelated data points as changes in a series lower down the stack will have a difficult to gauge effect on the series above it.
	Data table The data table provides a detailed breakdown, in tabular format, of the results of a composed aggregation. Tip, a data table is available from many other charts by clicking grey bar at the bottom of the chart.
	Line chart Often the best chart for high density time series. Great for comparing one series to another. Be careful with sparse sets as the connection between points can be misleading.
	Markdown widget Useful for displaying explanations or instructions for dashboards.
	Metric One big number for all of your one big number needs. Perfect for show a count of hits, or the exact average a numeric field.
	Pie chart Pie charts are ideal for displaying the parts of some whole. For example, sales percentages by department. Tip: Pie charts are best used sparingly, and with no more than 7 slices per pie.
	Tile map Your source for geographic maps. Requires an Elasticsearch geo_point field. More specifically, a field that is mapped as type:geo_point with latitude and longitude coordinates.
	Vertical bar chart The goto chart for oh-so-many needs. Great for time and non-time data. Stacked or grouped, exact numbers or percentages. If you are not sure which chart your need, you could do worse than to start here.

Kibana: Combine visualizations to a Dashboard



Some use cases of ELK Stack

- Log data management and analysis
- Monitor systems and / or applications and notify operators about critical events
- Collect and analyze other (mass) data
 - i.e. business data for business analytics
 - Energy management data or event data from smart grids
 - Environmental data
- Use the ELK stack for search driven access to mass data in web based information systems

Log Data Management and Analysis

Many different types of logs

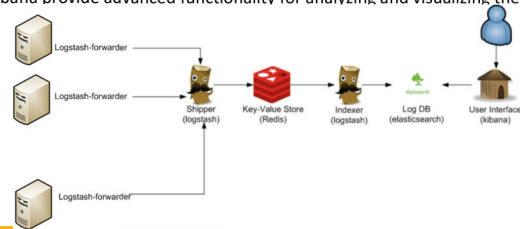
- Application logs
- Operating system logs
- Network traffic logs from routers, etc.

Different goals for analysis

- Detect errors at runtime or while testing applications
- Find and analyze security threats
- Aggregate statistical data / metrics

ELK provides solutions

- Logstash allows to collect all log entries at a central place (e.g. Elasticsearch)
- End users don't need to know where the log files are located
- Big log files will be transferred continuously in smaller chunks
- Easy access for end users via Browser based interfaces (e.g. Kibana)
- Elasticsearch / Kibana provide advanced functionality for analyzing and visualizing the log data



Splunk vs. ELK

Comparison criteria	Splunk	ELK (ElasticSearch, Logstash and Kibana)
Technology used for indexing	C++ based proprietary	Java based Apache Lucene
Technology used for searching	MapReduce based	Apache Lucene based
Language used for search	Splunk Processing Language	Query DSL
REST API for search interface	Available	Available

IMP Note to Self



Thank you



BITS Pilani
Pilani | Dubai | Goa | Hyderabad | Mumbai
**WORK INTEGRATED
LEARNING PROGRAMMES**



BITS Pilani
Pilani Campus

Introduction to DevOps

Slides edited /compiled by Harvinder S Jabbal
for DevOps faculty team.

CSIW ZG515, Introduction to DevOps CS 12 : Configuration Management

IMP Note to Self



IMP Note to Students

- It is important to know that just login to the session does not guarantee the attendance.
- Once you join the session, continue till the end to consider you as present in the class.
- IMPORTANTLY, you need to make the class more interactive by responding to Professors queries in the session.
- **Whenever Professor calls your number / name ,you need to respond, otherwise it will be considered as ABSENT.**

BITS Pilani, Pilani Campus

